

Selecting Model Complexity for Runge’s Function: Regularized Regression, Resampling and Gradient Descent

Sigurd Riseth
Department of Physics
University of Oslo
Oslo, Norway
sigurri@uio.no

Abstract—Runge’s function is the classic example of polynomial interpolation on equispaced points diverging as the degree increases, which makes it a demanding test of how model complexity should be selected. We fitted polynomials of degree up to 15 to noisy samples of the function using ordinary least squares (OLS), Ridge and Lasso regression, and estimated the test error with the bootstrap and k-fold cross-validation. For OLS, the test mean squared error was minimized at degree d ; beyond this degree, the bootstrap variance grew d -fold by degree d . Cross-validation selected [Ridge] at degree d and penalty λ , with a test error λ % below that of the best OLS model. We then replaced the closed-form solutions with our own gradient-descent optimizers, using both analytical and automatically differentiated gradients. [Adam] reached the closed-form solution to a relative error of ϵ in n iterations, n times fewer than plain gradient descent. Lasso, which has no closed-form solution, [set λ of λ coefficients to zero without improving on Ridge]. [For smooth targets such as this, shrinkage controlled the variance more effectively than sparsity.]

Index Terms—regression, regularization, resampling, bias-variance trade-off, gradient descent, automatic differentiation

I. INTRODUCTION

Fitting a function with a polynomial is the simplest non-trivial regression problem, and it is not a trivial one. The Weierstrass approximation theorem guarantees that any continuous function on a closed interval can be approximated uniformly to arbitrary accuracy by polynomials, but it says nothing about how to find such a polynomial from a finite set of noisy samples, or which degree to choose. Increasing the degree reduces the approximation error of the model class, yet it also makes the fitted coefficients increasingly sensitive to the particular data set. This tension is the bias-variance trade-off [1], [2]. Its lessons carry over essentially unchanged to the more flexible models used elsewhere in machine learning, which makes polynomial regression a transparent laboratory for them.

Runge’s function,

$$f(x) = \frac{1}{1 + 25x^2}, \quad x \in [-1, 1], \quad (1)$$

is the classical example of how this can fail. Although f is smooth, its polynomial interpolants on equispaced nodes

diverge near the endpoints as the degree increases; this is Runge’s phenomenon [3], [4]. Least-squares regression on noisy data is not interpolation, but it faces the same underlying difficulty in a statistical form. A high-degree fit on equispaced points is free to oscillate between the data, it fits the noise as readily as the signal, and the monomial design matrix becomes severely ill-conditioned. Runge’s function therefore provides a demanding test of how model complexity should be selected and controlled.

We study three linear models. Ordinary least squares (OLS) provides the unregularized baseline, whose test error as a function of degree exposes the trade-off directly. Ridge regression [5] and Lasso regression [6] add an ℓ_2 and an ℓ_1 penalty, respectively, trading a controlled increase in bias for a reduction in variance. The Ridge penalty shrinks all coefficients, while the Lasso penalty can set some of them exactly to zero. To estimate the test error and select the degree and penalty, we use two resampling methods: the bootstrap, which we also use to decompose the error into bias and variance, and k -fold cross-validation.

OLS and Ridge have closed-form solutions, whereas Lasso does not, since its penalty is not differentiable at zero. We therefore also solve all three problems iteratively, using plain gradient descent, momentum, AdaGrad, RMSprop and Adam [7], as well as stochastic gradient descent with mini-batches. The closed-form OLS and Ridge solutions serve as exact benchmarks for these optimizers. We compute gradients both analytically and by automatic differentiation [8], and verify that the two agree.

All regression models, the bootstrap, the gradient-descent optimizers and the analytical gradients are our own implementations. Cross-validation uses the `KFold` and `cross_validate` utilities of `scikit-learn` [9], automatic differentiation uses `JAX` [10], and our Lasso solver is compared against `scikit-learn`’s Lasso.

The remainder of this report is structured as follows. Section II presents the theoretical background: linear regression, the bias-variance decomposition, and the resampling and gradient-descent methods. Section III describes the implementation and how it was verified. Section IV presents and

discusses the results, and section V summarizes our findings and outlines future work. Appendix A declares our use of large language models. All code and selected results are available at <https://github.com/SigurdRiseth/FYS-STK4155-P1>.

II. MATHEMATICAL FOUNDATIONS

This section presents the mathematical methods used in this study. We first introduce linear regression via ordinary least squares, followed by regularization through Ridge and Lasso. We then discuss the bias–variance trade-off, followed by the resampling methods bootstrap and k -fold cross-validation, and conclude with gradient descent and automatic differentiation. We assume familiarity with basic linear algebra and statistics.

A. Linear regression

Linear regression is among the earliest techniques in machine learning. It addresses the scenario in which we are given n samples with p features, arranged in a design matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$, along with n targets $\mathbf{y} \in \mathbb{R}^n$. We assume the targets are produced according to

$$\mathbf{y} = f(\mathbf{x}) + \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \sigma^2), \quad (2)$$

where f denotes an unknown function, $\boldsymbol{\epsilon}$ is the noise vector, and σ^2 is the noise variance. The goal is to use the observed data to build an estimator $\hat{\mathbf{y}}$ that approximates f .

We can define a design matrix as $\mathbf{X} \in \mathbb{R}^{n \times p}$, where n is the number of instances and p the number of parameters. We then also have $\boldsymbol{\theta} \in \mathbb{R}^p$, which is the vector of model parameters (weights). The predicted target values can then be expressed as $\tilde{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta}$, where $\tilde{\mathbf{y}} \in \mathbb{R}^n$.

1) *Singular value decomposition*: The singular value decomposition (SVD) is a powerful algorithm that allows us to decompose any matrix $\mathbf{X}$ into a diagonal matrix $\boldsymbol{\Sigma}$ and two orthogonal matrices, $\mathbf{U}$ and $\mathbf{V}$. The SVD can be expressed as

$$\mathbf{X} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top. \quad (3)$$

For a design matrix $\mathbf{X}$ of dimension $n \times p$, the matrix $\mathbf{U}$ is $n \times n$, $\mathbf{V}$ is $p \times p$, and $\boldsymbol{\Sigma}$ is $n \times p$ with $r = \min(n, p)$ singular values $\sigma_i \geq 0$ on the main diagonal and zeros elsewhere [2].

2) *Ordinary least squares*: In ordinary least squares (OLS), the model parameters $\boldsymbol{\theta}$ are chosen such that the predicted values $\mathbf{X}\boldsymbol{\theta}$ are as close as possible to the observed response vector $\mathbf{y} \in \mathbb{R}^n$ [11]. This is achieved by minimizing the mean squared error (MSE),

$$C_{\text{OLS}}(\boldsymbol{\theta}) = \frac{1}{n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2, \quad (4)$$

where the factor $1/n$ does not affect the minimizer, but makes the objective independent of sample size. Formally, we can denote the solution to this problem as

$$\hat{\boldsymbol{\theta}}_{\text{OLS}} = \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^p} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2, \quad (5)$$

which after straightforward differentiation leads to the closed form equation

$$\hat{\boldsymbol{\theta}}_{\text{OLS}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (6)$$

The objective is convex, since its Hessian $\frac{2}{n} \mathbf{X}^\top \mathbf{X}$ is positive semi-definite; any stationary point is therefore a global minimizer [2].

The OLS estimator can also be expressed as

$$\hat{\boldsymbol{\theta}}_{\text{OLS}} = \mathbf{X}^+ \mathbf{y}, \quad (7)$$

where $\mathbf{X}^+$ denotes the Moore–Penrose pseudoinverse, which can be computed using SVD [12]. This formulation is numerically more stable than solving the normal equations, since forming $\mathbf{X}^\top \mathbf{X}$ squares the condition number of $\mathbf{X}$, which can result in a substantial loss of numerical precision [2].

3) *Ridge regression*: By adding a regularizer defined as the L_2 norm of the parameter vector to the cost function, we achieve the penalized regression problem called Ridge regression [11]:

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}}(\lambda) = \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^p} \left(\|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_2^2 \right). \quad (8)$$

Differentiating with respect to $\boldsymbol{\theta}$ yields a matrix inversion problem similar to that of eq. (6), but with a crucial difference: for any finite λ , the added term removes the singularity issues that OLS can encounter. The optimal parameters are then given by

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}}(\lambda) = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (9)$$

This objective is strictly convex, meaning the minimum is unique, as the Hessian $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$ is positive definite for all $\lambda > 0$ [2].

Just as with OLS, the Ridge prediction can be written using the SVD,

$$\tilde{\mathbf{y}}_{\text{Ridge}} = \left(\sum_{j=0}^{p-1} d_j \mathbf{u}_j \mathbf{u}_j^\top \right) \mathbf{y}, \quad (10)$$

where each direction $\mathbf{u}_j$ is scaled by the shrinkage factor

$$d_j = \frac{\sigma_j^2}{\sigma_j^2 + n\lambda} \in (0, 1]. \quad (11)$$

If $n\lambda \gg \sigma_j^2$ for some direction j , then $d_j \approx \sigma_j^2 / (n\lambda) \approx 0$, and that direction is almost completely suppressed. Conversely, if $n\lambda \ll \sigma_j^2$, the factor d_j in eq. (11) is just below one, and the direction is essentially retained as in OLS.

4) *Lasso regression*: If we instead choose the L_1 norm for the regularizer, we obtain Lasso regression, defined as [2]

$$\hat{\boldsymbol{\theta}}_{\text{Lasso}}(\lambda) = \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^p} \left(\|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_1 \right). \quad (12)$$

Lasso stands for least absolute shrinkage and selection operator. Lasso has no closed form solution, and can thus not be solved by a matrix inversion such as eq. (6) and eq. (9) [2].

B. Bias–variance decomposition

To decompose the bias–variance trade-off, we assume the data-generating process of eq. (2). The prediction $\tilde{\mathbf{y}}$ is a random quantity because the parameters are fitted on a training set that is itself random [2]. All expectations below are taken jointly over the noise $\boldsymbol{\epsilon}$ on the test targets and over the draw

of the training set (equivalently, in the bootstrap, over the resampled training sets).

Starting from the cost we wish to decompose (MSE) and adding and subtracting $\mathbb{E}[\tilde{\mathbf{y}}]$ inside the square,

$$\begin{aligned}\mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] &= \mathbb{E}[(\mathbf{f} + \varepsilon - \tilde{\mathbf{y}})^2] \\ &= \mathbb{E}[(\mathbf{f} - \mathbb{E}[\tilde{\mathbf{y}}]) + (\mathbb{E}[\tilde{\mathbf{y}}] - \tilde{\mathbf{y}}) + \varepsilon]^2. \quad (13)\end{aligned}$$

Writing $a = \mathbf{f} - \mathbb{E}[\tilde{\mathbf{y}}]$, $b = \mathbb{E}[\tilde{\mathbf{y}}] - \tilde{\mathbf{y}}$ and $c = \varepsilon$, the square expands into three squared terms and three cross terms,

$$\begin{aligned}\mathbb{E}[(a + b + c)^2] &= \mathbb{E}[a^2] + \mathbb{E}[b^2] + \mathbb{E}[c^2] \\ &\quad + 2\mathbb{E}[ab] + 2\mathbb{E}[ac] + 2\mathbb{E}[bc]. \quad (14)\end{aligned}$$

Consider the three cross terms:

$$\mathbb{E}[ab] = (\mathbf{f} - \mathbb{E}[\tilde{\mathbf{y}}]) \mathbb{E}[\mathbb{E}[\tilde{\mathbf{y}}] - \tilde{\mathbf{y}}] = 0, \quad (15)$$

$$\mathbb{E}[ac] = (\mathbf{f} - \mathbb{E}[\tilde{\mathbf{y}}]) \mathbb{E}[\varepsilon] = 0, \quad (16)$$

$$\mathbb{E}[bc] = \mathbb{E}[\mathbb{E}[\tilde{\mathbf{y}}] - \tilde{\mathbf{y}}] \mathbb{E}[\varepsilon] = 0, \quad (17)$$

where we used that a is deterministic, that $\mathbb{E}[b] = \mathbb{E}[\mathbb{E}[\tilde{\mathbf{y}}] - \tilde{\mathbf{y}}] = 0$, that $\mathbb{E}[\varepsilon] = 0$, and that ε is independent of both $\mathbf{f}$ and the training data (so it factors in $\mathbb{E}[bc]$). What remains is

$$\mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] = \underbrace{(\mathbf{f} - \mathbb{E}[\tilde{\mathbf{y}}])^2}_{\text{Bias}^2[\tilde{\mathbf{y}}]} + \underbrace{\mathbb{E}[(\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}])^2]}_{\text{Var}[\tilde{\mathbf{y}}]} + \sigma^2. \quad (18)$$

In practice we report the sample averages over n test points,

$$\text{Bias}^2[\tilde{\mathbf{y}}] = \frac{1}{n} \sum_i (f_i - \mathbb{E}[\tilde{y}_i])^2, \quad (19)$$

$$\text{Var}[\tilde{\mathbf{y}}] = \frac{1}{n} \sum_i (\tilde{y}_i - \mathbb{E}[\tilde{y}_i])^2, \quad (20)$$

with $\mathbb{E}[\tilde{y}_i]$ estimated by averaging the prediction at point i over the bootstrap replicates.

The three terms have distinct meanings. The *bias* measures how far the average model prediction is from the true function f : it is large when the model is too rigid to represent f (low polynomial degree). The *variance* measures how much the prediction fluctuates around its own mean as the training set changes: it grows with model complexity. The *noise variance* σ^2 is irreducible and sets a floor on the achievable test error [2].

Since f is unknown, we cannot evaluate the bias directly and replace $\mathbf{f}$ by the observed targets $\mathbf{y} = \mathbf{f} + \varepsilon$. The measured bias then becomes

$$\frac{1}{n} \sum_i \mathbb{E}[(y_i - \mathbb{E}[\tilde{y}_i])^2] = \text{Bias}^2[\tilde{\mathbf{y}}] + \sigma^2, \quad (21)$$

so the estimated bias absorbs the noise variance: the measured $\text{Bias}^2 + \text{Var}$ sums to the full test MSE, with σ^2 folded into the bias rather than appearing as a separate term [2].

C. Resampling methods

The standard situation in machine learning research is one where you have limited access to data – as opposed to our situation where we can easily generate more as we know f .

Resampling methods are statistical techniques that repeatedly draw new random samples from an existing dataset.

1) Bootstrap:

2) *k-fold cross-validation*: In k -fold cross-validation, the dataset is partitioned into k folds, and the model is trained k separate times, each time leaving out a different fold for evaluation [13].

This is commonly used in order to select hyperparameters, such as the penalty parameter λ in Ridge regression, in which you choose the value that minimize the average estimate of the prediction error over the folds [2].

D. Automatic differentiation and backpropagation

E. Gradient descent

For optimization problems where an analytical (closed-form) solution is unavailable, numerical optimization methods can be used to iteratively approach a minimum of the cost function. One such method is Newton's method, which uses both the gradient and the Hessian:

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \mathbf{H}^{-1} \nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}^{(k)}). \quad (22)$$

However, Newton's method requires computing and manipulating the Hessian. For a model with p parameters, the Hessian contains p^2 entries and inverting it costs $\mathcal{O}(p^3)$, making it impractical to store and manipulate for large-scale models [2].

Gradient descent provides a computationally simpler alternative by updating the parameters using only the gradient:

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \gamma \nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}^{(k)}), \quad (23)$$

where $\gamma > 0$ is the learning rate which replaces the Hessian. This makes gradient descent particularly suitable for large-scale machine-learning models, such as neural networks, where the cost function is generally nonlinear and does not have a closed-form solution.

1) *Momentum*: The convergence of gradient descent can be improved by adding a momentum term, which carries over part of the previous parameter update. The resulting update rule is

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \gamma \nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}^{(k)}) + \delta [\boldsymbol{\theta}^{(k)} - \boldsymbol{\theta}^{(k-1)}], \quad (24)$$

where $\delta \in [0, 1]$ is the momentum parameter. This is known for accelerating learning, especially in of high curvature, small but consistent gradients, or noisy gradients. Momentum is mainly intended to address two issues: ill-conditioning of the Hessian matrix and variance in the stochastic gradient [12].

2) *Stochastic gradient descent*: For a cost $C(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n c_i(\boldsymbol{\theta})$, the full gradient requires a pass over all n data points per iteration, which is expensive for large n [2]. Stochastic gradient descent (SGD) instead uses a mini-batch $\mathcal{B}_k$ of m examples i.i.d. sampled from the data, giving the unbiased gradient estimate [12]

$$\mathbf{g}^{(k)} = \frac{1}{m} \sum_{i \in \mathcal{B}_k} \nabla_{\boldsymbol{\theta}} c_i(\boldsymbol{\theta}^{(k)}), \quad \boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \gamma \mathbf{g}^{(k)},$$

where $\mathbf{g}^{(k)}$ is the gradient of the cost function at iteration k .

The sampling noise prevents SGD from converging exactly to the minimum; instead, the iterates fluctuate around it with

a spread scaling as $\sqrt{\gamma/m}$. This can be reduced by increasing m , at a higher cost per iteration, or by gradually decreasing γ during training, known as learning rate scheduling [2].

3) *AdaGrad*: AdaGrad, introduced by Duchi et al. [14], adapts the step size of each coordinate individually by accumulating the squared gradients,

$$\mathbf{r}^{(k)} = \mathbf{r}^{(k-1)} + \mathbf{g}^{(k)} \odot \mathbf{g}^{(k)}, \quad \mathbf{r}^{(0)} = \mathbf{0},$$

where $\odot$ denotes the Hadamard product. The parameters are then updated as

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \frac{\gamma}{\sqrt{\mathbf{r}^{(k)} + \epsilon}} \odot \mathbf{g}^{(k)},$$

with the square root and division taken element-wise, and ϵ a small constant for numerical stability. Coordinate j thus has the effective step size given by

$$\frac{\gamma}{\sqrt{r_j^{(k)} + \epsilon}},$$

which decreases monotonically as gradients accumulate [2]. Coordinates with large past gradients therefore take smaller steps, while rarely updated coordinates take larger steps [12].

4) *RMSProp*: Since AdaGrad accumulates all past squared gradients, its effective step size decreases monotonically, which can halt learning prematurely. RMSProp addresses this by replacing the cumulative sum with an exponentially decaying average [12],

$$\mathbf{v}^{(k)} = \rho \mathbf{v}^{(k-1)} + (1 - \rho) \mathbf{g}^{(k)} \odot \mathbf{g}^{(k)}, \quad \mathbf{v}^{(0)} = \mathbf{0},$$

where $\rho \in [0, 1)$ is the decay rate, typically 0.9 or 0.99, giving an effective memory of about $1/(1 - \rho)$ iterations. The parameters are updated as

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \frac{\gamma}{\sqrt{\mathbf{v}^{(k)} + \epsilon}} \odot \mathbf{g}^{(k)},$$

with the square root and division taken element-wise [2].

5) *Adam*: [7]

III. IMPLEMENTATION

The problem solution was implemented as a Python project. This section aims to introduce this implementation, as well as decisions/trade-offs made and show how it was verified. The source code, exploratory notebooks, tests, and experimental scripts required to reproduce the results are available in the publicly hosted project repository [15].

A. Data generation

We generate noisy observations from Runge's function by adding i.i.d. Gaussian noise, as shown in eq. (2), where f is Runge's function (1) and the noise ϵ has standard deviation $\sigma = 0.1$. We generate the data points (x_i, y_i) by drawing n x -values uniformly from the interval $[-1, 1]$, evaluating f at each, and adding the noise. An example of the resulting data points is shown in fig. 1.

The data was subsequently split into training and test sets (80/20), before standardization. The split was performed before standardization to avoid data leakage. Standardization was

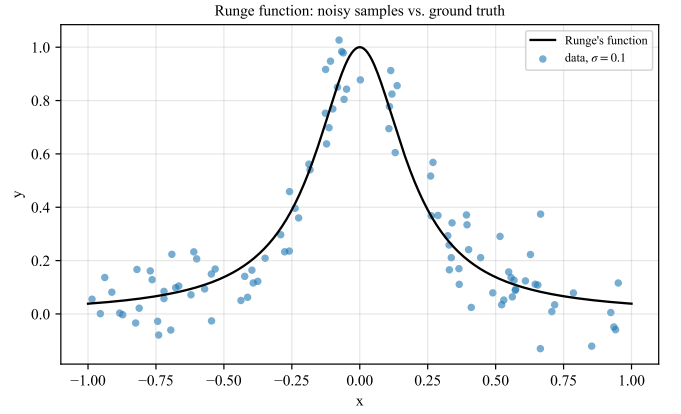


Fig. 1. Noisy samples ($n = 100$, $\sigma = 0.1$, seed 42) drawn from Runge's function, shown against the noise-free ground truth.

applied using `StandardScaler`, so that each column of the design matrix has mean zero and unit standard deviation, and centered target has mean zero. This improves the conditioning of the optimization landscape and ensures that the penalties in Ridge and Lasso regression treat each coefficient fairly, since the features are on the same scale [2], [13].

B. Code architecture

The source code was implemented in Python using the `numpy`, `sklearn`, and `jax` libraries [9], [10], [16]. For `jax`, the precision was changed to double precision.

1) *Regression*: The regression models all inherit from the `LinearModel` abstract base class, which follows the `sklearn` estimator convention with `fit` and `predict` functions, and hyperparameters set in the constructor. This lets our estimators be dropped directly into `sklearn` utilities such as `KFold` and `cross_validate`. The base class provides the shared `predict` method and input validation, while each subclass implements `fit`. OLS solves the least-squares problem via the SVD-based pseudoinverse, which remains stable for ill-conditioned or rank-deficient design matrices. Ridge instead solves the normal equations from eq. (9) directly with `numpy.linalg.solve`, since the penalty term guarantees a well-conditioned system, with the intercept optionally excluded from the penalty. Lasso, which lacks a closed-form solution, is fit via gradient descent.

2) *Resampling*: Divided into `bootstrap.py` and `cross_validation.py`, where `bootstrap` uses `numpy` and the cross-validation uses `sklearn`'s `KFold` class and `cross_val_score` function.

3) *Gradient descent*: The gradient descent code is implemented as its own package (`optimization`) containing all the different variants of gradient descent. They all roughly follow algorithm 1, but replace line 3 with their own corresponding update equation.

Algorithm 1 Basic gradient descent

Require: Initial parameters $\theta^{(0)}$, learning rate $\eta > 0$, tolerance $\varepsilon > 0$, maximum iterations $N_{\max}$
Ensure: Approximate minimizer θ^* of $J(\theta)$
1: $k \leftarrow 0$
2: **while** $\|\nabla J(\theta^{(k)})\| > \varepsilon$ **and** $k < N_{\max}$ **do**
3: $\theta^{(k+1)} \leftarrow \theta^{(k)} - \eta \nabla J(\theta^{(k)})$ ▷ Equation (23)
4: $k \leftarrow k + 1$
5: **end while**
6: **return** $\theta^{(k)}$

C. Verification

IV. RESULTS AND DISCUSSION

All results shown here are from $n = 100$ points drawn uniformly from $[-1, 1]$, noise level $\sigma = 0.1$, a 80/20 train–test split, and the predictors centered and scaled using statistics computed on the training set only, unless otherwise is stated.

A. Analytical solutions for OLS and Ridge

Figure 2 shows the training and test error of the OLS fit as a function of polynomial degree. The training error decreases monotonically with degree, while the test error flattens out and then rises, the qualitative signature of overfitting: past some complexity the added flexibility is spent on the noise realization rather than on f . For this particular split the test MSE is minimized at degree 12. The left panel reproduces the behavior of fig. 2.11 of Hastie et al. [1], although less clear.

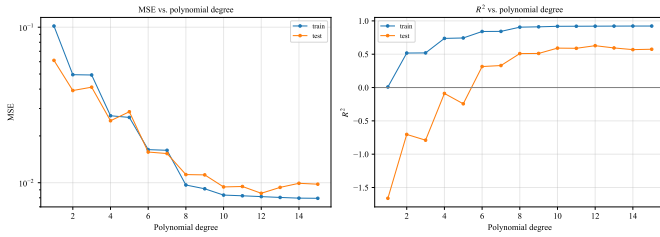


Fig. 2. Best OLS model at degree 12, where we see the model overfits (test error increases) at higher degrees. For $n = 100$ points drawn uniformly from $[-1, 1]$, noise level $\sigma = 0.1$, a 80/20 train–test split, and predictors centered and scaled using statistics computed on the training set only.

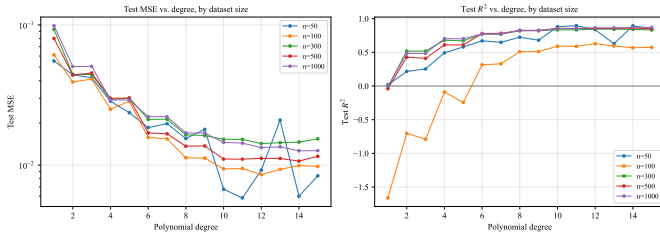


Fig. 3. Test MSE (left) and R^2 (right) for OLS vs. polynomial degree for varying training-set sizes

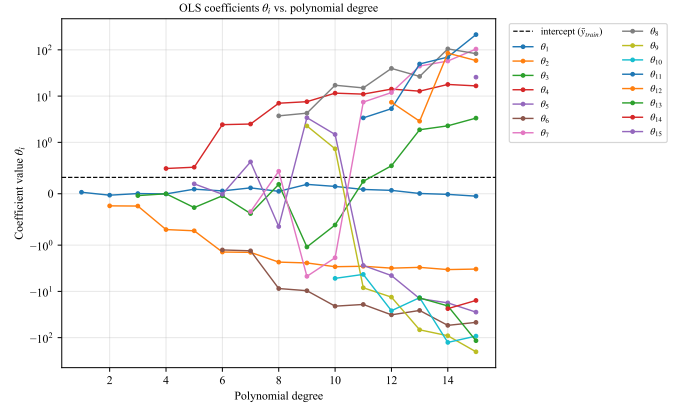


Fig. 4. Increasing polynomial degree leads to higher coefficient values (θ_i). The intercept is given by $\bar{y}_{\text{train}}$ due to the standardization performed.

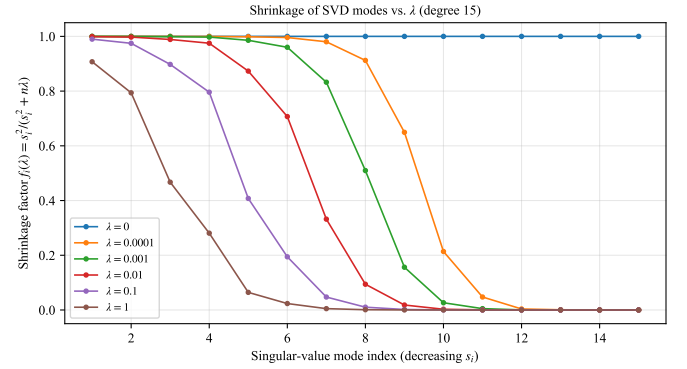


Fig. 5. Shrinkage of SVD modes vs. λ (degree 15). We observe the singular-values with a higher index (i.e., lower value) have a sharper shrinkage, effectively performing parameter selection. We also see that $\lambda = 0$ gives no shrinkage and is equivalent to OLS. This is linked to the shrinkage factor given in eq. (11).

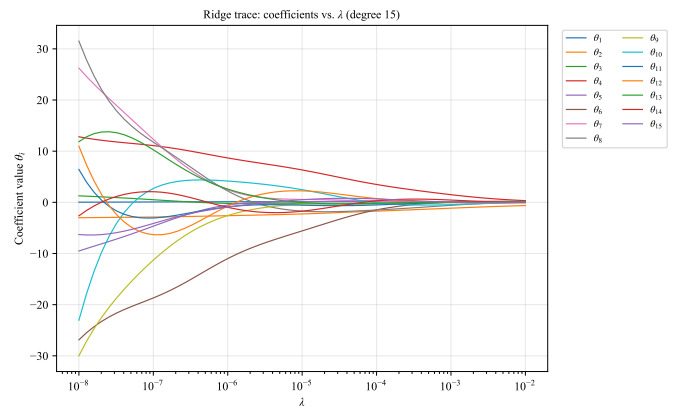


Fig. 6. Ridge trace: coefficients vs. λ (degree 15)

B. Bias–variance via resampling

Figure 7 shows the bias–variance decomposition from eq. (18), with bias estimated as in eq. (21), for the OLS estimators. Simple models are dominated by bias, while complex models are dominated by variance; the best-performing degree lies between the two regimes, where both terms are small and the test MSE nearly reaches the irreducible noise floor σ^2 .

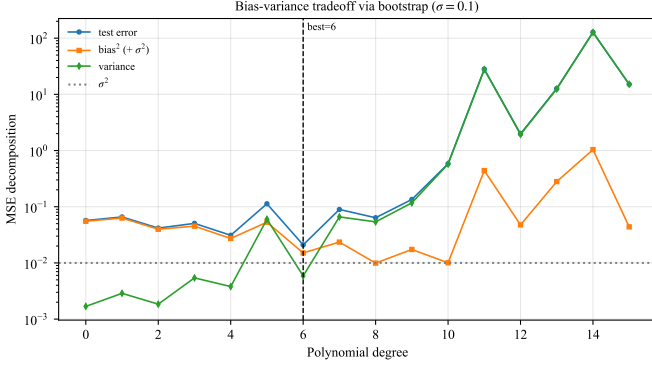


Fig. 7. Bias–variance trade-off via bootstrap

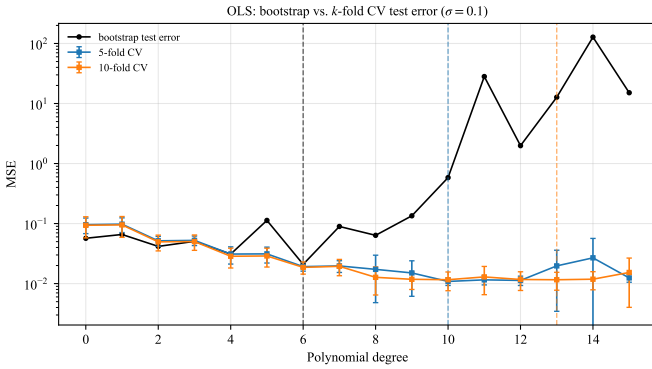


Fig. 8. Bootstrap vs k -fold cross-validation test MSE for OLS. We clearly see that for bootstrap, the error starts increasing after degree 6, while 5-fold has best degree at 10 and 10-fold has best degree at 13 (the highest degree checked).

C. Optimizer comparison

Linear regression makes an excellent benchmark for gradient descent for three reasons: it has a closed-form solution for comparison, its gradient can be calculated exactly, and its convex cost function ensures convergence as long as the learning rate is sufficiently small [2].

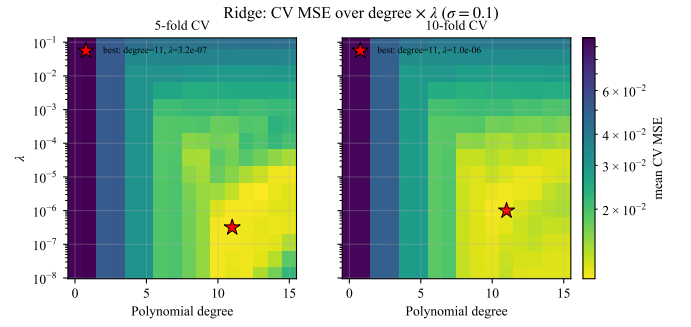


Fig. 9. Cross-validation MSE over degree and λ search for Ridge regression. We observe optimum at degree 11 and $\lambda = 3.2e - 07$ for 5-fold CV and degree 11 and $\lambda = 1.0e - 06$ for 10-fold CV.

V. CONCLUSION

REFERENCES

- [1] T. Hastie, R. Tibshirani, and J. H. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (Springer Series in Statistics), Second edition. New York, NY: Springer, 2017, ISBN: 978-0-387-84857-0.
- [2] M. Hjorth-Jensen, *Machine Learning and the Physical Sciences: From Discriminative Methods to Generative Methods and Reinforcement Learning*. Springer, Oct. 9, 2026. [Online]. Available: <https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/doc/BookML/Lecturebook.pdf>
- [3] C. Runge, “Über empirische Funktionen und die Interpolation zwischen äquidistanten Ordinaten,” *Zeitschrift für Mathematik und Physik*, vol. 46, pp. 224–243, 1901.
- [4] S. De Marchi, F. Marchetti, E. Perracchione, and D. Poggiali, “Polynomial interpolation via mapped bases without resampling,” *Journal of Computational and Applied Mathematics*, vol. 364, p. 112 347, Jan. 2020, ISSN: 03770427. DOI: 10.1016/j.cam.2019.112347 Accessed: Sep. 17, 2026. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0377042719303449>
- [5] A. E. Hoerl and R. W. Kennard, “Ridge Regression: Biased Estimation for Nonorthogonal Problems,” *Technometrics*, vol. 12, no. 1, pp. 55–67, Feb. 1, 1970, ISSN: 0040-1706. DOI: 10.1080/00401706.1970.10488634 Accessed: Sep. 22, 2026. [Online]. Available: <https://doi.org/10.1080/00401706.1970.10488634>
- [6] R. Tibshirani, “Regression Shrinkage and Selection Via the Lasso,” *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 58, no. 1, pp. 267–288, Jan. 1, 1996, ISSN: 0035-9246. DOI: 10.1111/j.2517-6161.1996.tb02080.x Accessed: Sep. 22, 2026. [Online]. Available: <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>

- [7] D. P. Kingma and J. Ba. “Adam: A Method for Stochastic Optimization.” arXiv: 1412.6980 [cs.LG], Accessed: Sep. 21, 2026. [Online]. Available: <http://arxiv.org/abs/1412.6980> pre-published.
- [8] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, “Automatic Differentiation in Machine Learning: A Survey,” *Journal of Machine Learning Research*, vol. 18, pp. 1–43, 2018. [Online]. Available: <https://jmlr.org/papers/v18/17-468.html>
- [9] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011. [Online]. Available: <http://jmlr.org/papers/v12/pedregosa11a.html>
- [10] J. Bradbury et al., *JAX: Composable transformations of Python+NumPy programs*, version 0.3.13, 2018. [Online]. Available: <http://github.com/jax-ml/jax>
- [11] P. Mehta et al., “A high-bias, low-variance introduction to Machine Learning for physicists,” *Physics Reports*, vol. 810, pp. 1–124, May 2019, ISSN: 03701573. DOI: 10.1016/j.physrep.2019.03.001 Accessed: Aug. 26, 2026. [Online]. Available: <http://arxiv.org/abs/1803.08823>
- [12] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. Accessed: Aug. 12, 2026. [Online]. Available: <https://www.deeplearningbook.org/>
- [13] A. Géron, *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems* (Data Science / Machine Learning), Third edition. O’Reilly, 2023, 834 pp., ISBN: 978-1-0981-2597-4.
- [14] J. Duchi, E. Hazan, and Y. Singer, “Adaptive Sub-gradient Methods for Online Learning and Stochastic Optimization,”
- [15] S. Riseth, *FYS-STK4155 Project 1*, version v0.1.0, GitHub, Oct. 2026. [Online]. Available: <https://github.com/sigurdriseth/FYS-STK4155-P1>
- [16] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 17, 2020, ISSN: 0028-0836, 1476-4687. DOI: 10.1038/s41586-020-2649-2 Accessed: Aug. 23, 2026. [Online]. Available: <https://www.nature.com/articles/s41586-020-2649-2>

APPENDIX

A. Use of AI/LLM Tools

Artificial intelligence and large language models (LLMs) have been used as tools throughout the development and writing of this project. This appendix describes how these tools were used and identifies the parts of the project in which they contributed. The declaration follows the guidelines provided by the course¹, refer to it for understanding the levels in the tables below.

¹https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/LLM_Usage_Declaration_Guidelines.md

1) *Tools used:* Below we summarize all AI tools used in the development and writing of this project.

- Anthropic’s Claude, accessed via <https://claude.ai/> and Claude Code, September–October 2026. Models used include Claude Sonnet 4 and Claude Opus 4.
- OpenAI’s ChatGPT, accessed via <https://chatgpt.com/>, September–October 2026.

2) *Text Writing and Editing:* For each section of the report, we state the role that LLMs have played in contributing in table I.

TABLE I
LLM CONTRIBUTION TO TEXT WRITING AND EDITING.

Section	Level	Description
Abstract	3	Draft generated from the other report sections; used as a starting point and substantially revised.
Introduction	2	Two paragraphs rephrased for clarity.
Methodology	1	Grammar checking and minor rephrasing only.
Results	1	Grammar checking and minor rephrasing only.
Discussion	1	Grammar checking and minor rephrasing only.
Conclusion	1	Grammar checking and minor rephrasing only.

For the level 3 contribution to the abstract, Claude Opus was prompted to draft an abstract based on the completed sections of the report. The generated text was subsequently reviewed, corrected, and substantially revised by the authors.

3) *Code Generation and Assistance:* For each code file and notebook submitted as part of the project, we state the role of LLMs in table II. Level 0 indicates that no LLM assistance was used.

Claude Code was also used extensively for writing docstrings and unit tests throughout the code base. These contributions are not listed individually in table II, as they were applied consistently across the code base.

B. Additional Results

Here we present additional results.

1) *Bias–variance trade-off:* Increasing the dataset size lowers the variance at any given degree: with more data the fit depends less on the particular training sample, so each prediction $\tilde{y}_i$ varies less around its mean and the variance term in eq. (20) shrinks. The bias curve – a property of the model family, not of n – barely moves. This postpones the variance explosion and lets model complexity grow further before overfitting sets in, as seen in fig. 10, where the optimal degree moves from 4 at $n = 50$ to 12 at $n = 400$.

Increasing the noise level σ^2 has the opposite effect on complexity: fig. 11 shows that the error floor rises and the variance term scales up, while bias² is essentially unchanged, so noisier data favor simpler models and the best degree shifts downward.

TABLE II
LLM CONTRIBUTION IN CODE WRITING

File / Notebook	Level	Description
design_matrix.py	0	
runge.py	0	
optimizers.py	3	Claude provided the framework for the abstract <code>Optimizer</code> class and assisted with input-shape validation.
schedules.py	4	Claude fully generated the file with the three learning rate schedule functions. We reviewed and verified it.
autodiff.py	2	Claude assisted with the JIT-compilation implementation.
base.py	2	Claude implemented inheritance from sklearn's <code>BaseEstimator</code> and <code>RegressorMixin</code> , allowing the linear models to be used in scikit-learn pipelines.
cost.py	-	
degree_sweep.py	3	Claude refactored this out of notebooks and generalized it. Based on code written by us.
gradient_descent.py	2	Claude implemented the internal learning rate scheduling.
lasso.py	-	
ordinary_least_squares.py	1	Claude suggested the <code>rcond</code> parameter and singular-value cutoff logic for the SVD-based pseudoinverse.
ridge.py	-	
shrinkage.py	-	
bootstrap.py	-	
cross_validation.py	-	
metrics.py	0	
A_ols.ipynb	-	
B_ridge.ipynb	-	
C-D_bias-variance.ipynb	-	
E-F_gradient_descent.ipynb	-	
G_lasso.ipynb	-	
H_sgd.ipynb	-	
I_final_analysis.ipynb	-	

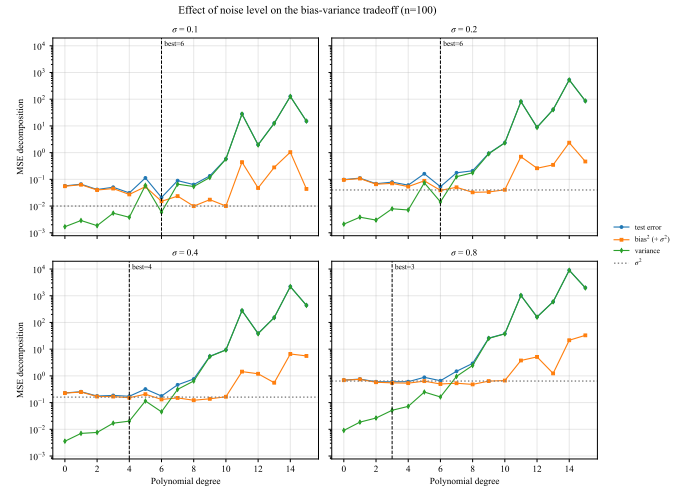


Fig. 11. Effect of sample size on the bias-variance trade-off

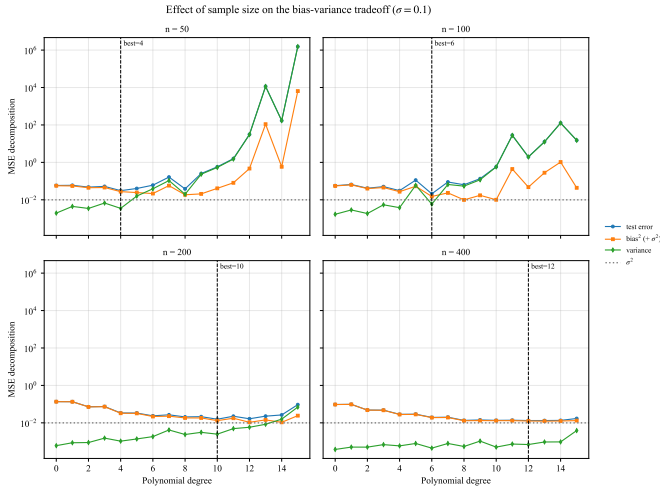


Fig. 10. Effect of sample size on the bias-variance trade-off